

The Agent Factory

A user tutorial for the three-phase, artifact-driven software pipeline

What you are looking at. The agent factory is a playbook for building software products with AI agents doing the work and humans holding the judgment. It is installed by copying one folder structure into a project repository. Product owners feed it ideas; specialist AI agents shape, plan, build, and verify; one human — the operator — decides what gets built and what gets merged.

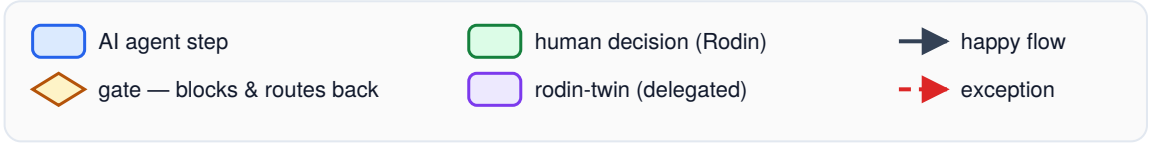
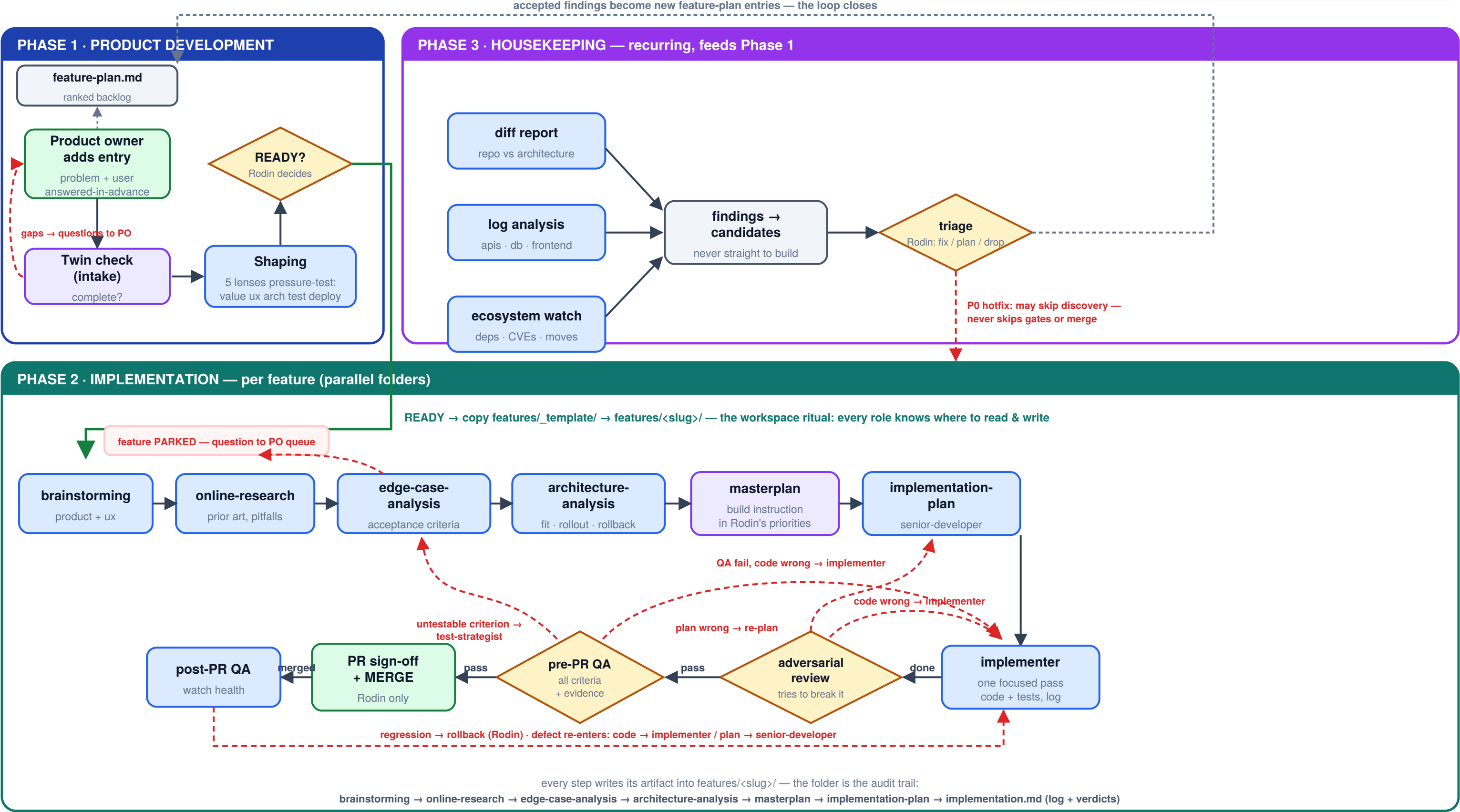
The one idea everything rests on. Agents never talk to each other. Every hand-off is a committed markdown file (an *artifact*) in a known location. A role reads the files its contract lists, does its job, and writes its own artifact for the next role. The folder is the audit trail: when something goes wrong, the files show exactly which step carried the error.

How to use this tutorial. Page 2 is the full process map — print it on A3 and keep it next to you. Pages 3–4 explain the concepts and the standing documents. Pages 5–9 walk through every step on the map, including all the exception paths. Page 10 gets you started.

The factory is duplicated per project: one repo, one substrate, one product. Multi-project coordination is deliberately out of scope.

AGENT FACTORY

Software product pipeline — three phases, artifact-driven, human at the irreversible gates

happy flowexception

Core concepts

Roles and authority

Every agent is defined by a markdown role document with a machine-readable header: what it reads, what it writes, who it hands off to, and — critically — its **authority level**:

Level	May do	May never do
advisory	Propose, pressure-test, recommend	Decide anything
gate	Block work and route it back, pass work that meets written criteria	Waive a criterion
executor	Act within an approved plan	Act outside the plan — stop and ask instead
sign-off (twin)	Approve within a written mandate	Merge, deploy, or decide anything reserved
monitor	Observe, report, recommend	Act

The gates

Two agent gates guard the build: the **adversarial reviewer** (tries to break each change) and **pre-PR QA** (verifies every acceptance criterion with evidence). Gates can stop and route work backwards, but only a human can waive a criterion. Three decisions are always human: marking a feature **Ready**, **merging** a pull request, and executing a **rollback**.

The twin

rodin-twin is not a persona — it is a mandate document, self-authored by the operator. It carries his ranked priorities, red lines, and an explicit list of what it may decide alone (delegated), what it drafts for him (advise-only), and what it must never touch (reserved). The twin checks work at two points — after a product owner adds an entry, and it authors each feature's masterplan. Features may run unattended all the way to an open, green pull request. **The twin never merges.**

The red lines

Four rules no agent, gate, or twin may waive — crossing one stops work until the operator decides:

- No new dependency without an ADR (architecture decision record).
- No PII or secrets in code, logs, or artifacts.
- No merge with red CI — ever, for any reason.
- Agents never touch production data.

The foundation documents

Four standing files anchor everything (the grey bar at the bottom of the map). Each kind of truth lives in exactly one place; every other file cites it instead of restating it.

substrate.md — the platform contract

What features are built *on*: the stack with pinned versions, repo layout, shared services, the data-model spine, coding conventions, the red lines — and the **Commands** section: install, build, lint, typecheck, test, run. Agents verify their own work by running these; “the full local check” means lint + typecheck + test + build, all green. Every substrate change is an ADR approved by the operator.

architecture.md — the living system design

How the system *is* shaped: components and their boundaries, data flow, cross-cutting concerns, the observability contract (what Phase 3 watches), and the append-only ADR log. When a feature changes the shape of the system, this file is updated in the same pull request.

feature-plan.md — the front door

The ranked backlog. Each entry carries the product owner’s voice so the pipeline rarely needs to call them back:

Field	Why it exists
Problem / user	Who hurts and how — never just the solution
PO (owner)	Who proposed it and answers product questions
Scope: in / explicitly out	The boundary that keeps the diff honest
Depends on / Shared work needed	Cross-feature order; shared things get built once, first — the guard against two parallel features building the same service twice
Answered in advance	The PO’s calls on predictable questions, written while they are present: out-of-scope, the constraint that matters most, what “good enough for v1” means
Open questions → PO	The async queue — agents append, answers land back. An unanswered question parks the entry

rodin-twin.md — the operator’s mandate

Priorities ranked (correctness → speed → small scope → visible value), red lines, free delegations (code style, library choice within existing dependencies, test depth, UI copy), the escalation format (options + trade-offs + recommendation + what is blocked), and the autonomy ceiling: unattended to PR-ready, never merged. Only the operator edits this file.

Phase 1 — Product development

The blue band on the map. Purpose: decide *what* to build and *why*, before any code.

Step 1 · Product owner adds an entry

The PO's only step in the whole factory. They state the problem and the user (not the solution) and fill **Answered in advance** while they are present. A PO can do this in a chat session with the product-strategist agent, which interviews, reframes, and formats the entry. Output: an entry in feature-plan.md with status *Idea*.

Step 2 · Twin check (intake)

Immediately after an entry is added or changed, the twin checks completeness: problem stated, hypothesis testable, Answered-in-advance filled, no substrate conflict. Two outcomes only — **complete**: marked twin-checked, proceeds silently; **gaps**: the twin writes precise questions into the entry's PO queue and flags the operator, whose only job is forwarding them.

Exception (red arrow on the map): the twin never guesses missing product intent and never rejects an entry — gaps always become questions.

Step 3 · Shaping — five lenses

Five advisory agents pressure-test the candidate, each through its own lens, writing concerns into the entry itself:

Agent	Lens	Typical question
product-strategist	Value & scope	Is this the smallest version that tests the hypothesis?
ux-strategist	The human using it	What are the states, errors, and dead ends?
architect	Fit & coherence	Where does this land; what does it strain?
test-strategist	Verifiability	How will we know it works?
deployment-strategist	Shippability	How does it roll out — and back?

Step 4 · READY? — the operator decides

The strategist proposes and ranks ($\text{impact} \times \text{confidence} \div \text{effort}$); the twin checks the Definition of Ready; **the operator marks Ready and sets the order**. Definition of Ready: problem and user stated · Answered-in-advance filled · scope has an in/out line · respects the substrate (or a signed ADR) · success is measurable · twin-checked · operator has marked it Ready. On Ready, the green arrow fires: a feature folder is born.

Phase 2 — Discovery to plan

The teal band, top row. One feature at a time enters; several features may run in parallel, each in its own folder.

The workspace ritual

First act of every feature: **copy features/_template/ to features/<slug>/** and set the feature reference in each file's header. This one copy is the contract that binds the whole crew: every role's reads and writes point into this folder, so all agents know where to read, what to do, and where to write. Parallel features never touch each other's artifacts.

The four discovery documents

Artifact	Authors	What it answers
brainstorming.md	product + ux strategists	What could this be? The shape of the solution space, and the shape we choose
online-research.md	any role	How have others solved it? Libraries, prior art, pitfalls to avoid
edge-case-analysis.md	test + ux strategists	Where does it break? Failure modes — and the acceptance criteria the rest of the pipeline verifies against
architecture-analysis.md	architect + deployment strategist	How does it fit and ship? Boundaries touched, rollout and rollback path, observability

Each document cites the one before it — the trail from idea to code stays unbroken.

masterplan.md — the twin's build instruction

When discovery is complete, **rodin-twin** synthesizes it into the build instruction, in the operator's priorities: what this build must deliver, the 2–4 key constraints in priority order, the build order within the feature (riskiest first, so surprises come early), and sequencing notes. One masterplan per feature; cross-feature order lives in the feature plan, not here.

implementation-plan.md — the senior developer's plan of change

The senior-developer turns the masterplan into concrete engineering: which files change and why (each traced to a criterion), the approach in small testable steps, any pre-refactor ("the change that makes the change easy" — done first, or explicitly "none"), the test surface mapping every criterion to a test, explicit out-of-scope, and rejected alternatives — the implementer inherits the reasoning, not just the verdict.

Exception (red, top of the band): if a blocking PO question is still unanswered when discovery needs it, the feature is PARKED — the question goes to the PO queue and the crew moves to another feature. Silence is never consent.

Phase 2 — Build and the two gates

The teal band, bottom row — read right to left on the map.

implementer — one focused pass

A fresh session that reads the implementation plan, the task at hand, and the substrate — and executes. Code plus tests, matching surrounding conventions, running the full local check until green. Everything is logged to `implementation.md`: what was done, what was skipped and why, command output as evidence. If the plan proves wrong mid-build, the implementer hands back to the senior-developer — it never improvises around a broken plan.

Gate 1 · adversarial review

A separate agent whose job is to **break** the change: hostile inputs, boundary violations, missing coverage of the acceptance criteria, failure paths, security. Findings are ranked blocker / major / minor. It reads the implementation plan too — because its verdict must say *what broke*:

Fail routing — the rule both gates follow

What the finding shows	Where the work goes back to	Why
The code is wrong (plan is sound, execution is not)	implementer	A coding mistake needs a coder, not a re-plan
The plan is wrong or incomplete (correct code could not fix it)	senior-developer — the implementation plan is revised	Patching around a broken plan is how drift starts
An acceptance criterion is untestable	test-strategist — edge-case-analysis is revised	The contract itself is defective

These are the dashed red arrows in the middle of the map: code-level failures loop to the implementer; plan-level failures climb back up to the planning row.

Gate 2 · pre-PR QA

When the build is done, an independent verifier checks the **whole** feature: fresh checkout builds clean, full suite green, every acceptance criterion demonstrated *with evidence* (command output, screenshots), substrate red lines spot-checked, no secrets, the diff scoped to the plan. It may not waive any criterion — waivers come from the operator. Fails route by the same table above.

Phase 2 — Shipping and watching

PR sign-off + merge — the operator's gate

The promoter composes the pull request as a story: what and why (linking the artifact trail), how to verify, rollout and rollback, risk. Within its mandate the twin may sign off *opening* the PR when every gate is green — features can reach this point unattended. **Merging is the operator's alone.** Approval is per-PR; one approval is never standing approval for the next.

post-PR QA — watch health

After the merge: verify the rollout completed, watch the health signals through the agreed window, confirm the success metric moves (or record its baseline). The feature-plan entry moves to Shipped / Observing.

Exception · regression → rollback

If something regressed (the long dashed arrow along the bottom of the map): post-PR QA recommends rollback, **the operator executes it** using the rollback path written in *architecture-analysis.md* — an executable runbook, not a research project. The defect then re-enters the flow by the same routing rule: code wrong to the implementer, plan wrong to the senior-developer.

All exception paths at a glance

Exception	Trigger	Route
Intake gaps	Entry incomplete at twin check	Questions to the PO queue; operator forwards
Feature parked	Blocking PO question unanswered	Feature waits; crew switches feature
Code wrong	Gate finding: execution error	Back to implementer
Plan wrong	Gate finding: plan cannot succeed	Back to senior-developer (plan revised)
Untestable criterion	QA cannot verify a criterion	Back to test-strategist (criteria revised)
Red-line contact	Any step touches a red line	Stop; ADR + operator decision
Regression in production	post-PR QA sees a health breach	Rollback (operator) + defect re-enters flow
P0 incident	Phase 3 finds something urgent	May skip discovery — never skips gates or merge

Phase 3 — Housekeeping

The purple band. Recurring, mostly autonomous observation that keeps the factory honest — and closes the loop back to Phase 1.

The three watchers

Role	Watches	Typical finding
diff report	What actually changed in the repo vs. what architecture.md claims	"A new service appeared that the architecture doc doesn't know about"
log analysis (apis · db · frontend)	Signals against the observability contract	"Error rate on /orders doubled since Tuesday's merge"
ecosystem watch	The outside world: dependency advisories, CVEs, ecosystem moves	"Our auth library has a critical CVE; patched in v4.2"

Findings become candidates — never builds

Every finding is written as a **candidate** feature-plan entry tagged "Source: Phase 3". Watchers propose; they never start work. The operator triages: fix now, schedule into the plan, or drop. Accepted findings enter Phase 1 like any other idea — the grey dashed arrow across the top of the map is the loop closing.

The P0 fast lane

A genuinely urgent finding (production down, exploited CVE) may be marked P0 by the operator at triage. A P0 may skip discovery — but the red arrow on the map lands *inside* Phase 2, not past it: it never skips the gates, and it never skips the operator's merge. Speed changes the path, not the authority.

Why Phase 3 matters

Without it, the factory only ever adds. The watchers are the factory's memory and conscience: drift gets named, regressions get caught while small, and the outside world's changes become planned work instead of surprises. A healthy rhythm: diff report weekly, log analysis on the observability window after every merge plus weekly, ecosystem watch monthly.

Getting started

Installing the factory

Run **start_prompt.md** (in the playbook root) with a capable agent session in your repository. It installs the playbook, interviews the operator to fill the twin, extracts and *verifies* the substrate from the codebase (every command is run before it is written down), baselines the architecture, opens the feature plan, configures the runtime (fills `factory.config.yaml`, sets up `.env` with the Claude subscription token from **claude setup-token** and the e2b key, smoke-tests the harness), runs an integrity check, and ends with a handover report. Expect 30–60 minutes with the operator present. How agents execute is config, not process: sandbox e2b, harness claude-code (switchable to pi in one line), models per role — default haiku, upgraded per role when quality demands it. Design in `runtime-plan.md`.

The first feature

- Pick the smallest, most boring feature available — feature #1 tests the factory, not the product.
- A PO (or the operator wearing the PO hat) writes the entry, including Answered-in-advance.
- Operator marks it Ready · copy the template folder · run discovery → masterplan → plan → build → gates → PR.
- Log every creak. Fix the playbook in a maintenance session (CONTEXT.md + the current zip) before feature #2.

Glossary

Term	Meaning
Artifact	A committed markdown file that carries work between roles — the only channel agents share
Role	An agent defined by a doc: mission, inputs, outputs, process, quality bar, guardrails, authority
Gate	A role that may block and route back, but never waive
Twin	The operator's self-authored mandate agent; signs within its mandate, escalates the rest
Operator	The one human running the pipeline (Rodin); owns Ready, merge, ADRs, waivers, rollback
PO	Product owner — contributes entries at intake; answers questions async via the entry
Red line	A rule no agent or twin may waive; crossing it stops work until the operator decides
ADR	Architecture decision record — how substrate and architecture changes are made and logged
Workspace ritual	Copying <code>features/_template/</code> to <code>features/<slug>/</code> — binds all roles to one feature
Full local check	lint + typecheck + test + build, all green — the loop every agent runs
Parked	A feature waiting on an unanswered PO question; silence is never consent
P0	Operator-declared urgent work; may skip discovery, never gates or merge

Version: playbook as of 2026-09-05 · Companion files: `process-flow-a3.svg` (this map) · `start_prompt.md` (setup) · `runtime-plan.md` + `factory.config.yaml` + `.env.example` (runtime) · `CONTEXT.md` (maintenance sessions)